

when to use @Component

@Component

Spring-managed object

Created, configured and used
by Spring

Life Cycle manage by Spring

Application Context

Ex

Request → Controller →

→ Repository

What is a Bean?

Can't annotate third-party classes.

Need custom creation/configuration.

Require init/destroy callbacks.

Configuring external services
(e.g. HTTP client)

Component vs. Bean

@Component

- Used for auto-detecting classes
- Register using classpath scanning
- used for your own classes

@Bean

- Manual declaration
- Used in configuration class
- Used for third party code.
- Conditional configuration

Applications Control

Control contains managing
authentication, cookies, and messages
bean lifecycle.
Also handles requests and responses

Spring A Shaded

Scopes

Singleton

(by default) (prototype)

Prototype

Web only:

Request

Session

Application

Websocket

Prototype

unique instance per request

Per-use bean.

stateless

WebScopes

Request

Session

Application

websocket

Enabled using :

- @ WebMvcTest
- @ SpringBootTest

why Environment-aware Config Matters

Different behavior for dev, test and prod.

Toggle feature with out code changes

Load specific beans based on system state

clean Separation of concerns

Spring profiles

logical grouping of Beans
around which env you are working
& working with.

uses @Profile annotation

spring.profiles.active property

Beans Conditional Annotations

@Conditional on property

@Conditional on Missing Bean

@Conditional on class

Custom Conditional Logic

Implements Condition interface

Evaluate Condition Programmatically

Usage @Conditional (CustomCondition.class)

Initialization Techniques

Using Lazy

Factory Method

Custom @ Component

Using @ Lazy

Deferred bean creation

Startup time

Initialize on request.

Custom @ Component

@ Lazy

Logging

Aspect oriented programming.